第 1 讲：整数与汇编入门

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1. 先知道我们正在看什么
2. 二进制和十六进制：先会读位串
3. 位运算和逻辑运算：长得像，处理对象不同
4. 无符号与补码：理解负数为什么这样存
5. C 类型转换：先定类型，再算数
6. 运算、溢出与移位
7. 字节序：变的是存放顺序，不是数值
8. 汇编入门：先读宽度，再读源与目的
9. 独立练习：6 道短题 + 1 段追踪
10. 达标后的 2 道可选加练
11. 收束：做题时按这个顺序检查
12. 对照来源与回查位置

本讲对应复习第 1 天，覆盖课程第 2 讲及第 4 讲的基础，不是课程第 1 讲。先读懂“同一串位怎样被解释”，再读懂“一条指令究竟改变了什么”。这两个问题解决后，后面的条件、循环、调用和数组就有了落脚点。

本讲的文字、图和教学推导为助手整理；标为“助手补充”的例题与练习是教学自编。真题改编会单独注明出处。独立题答案在第 1 讲练习答案，请写完自己的尝试再打开。

1. 先知道我们正在看什么

1.1 程序、CPU、寄存器与内存

C 程序里的 `x = x + 3;` 描述一件事：取得原来的 x，加上 3，保存结果。CPU 不直接阅读这句 C，而是执行编译器生成的机器指令。汇编是机器指令的文字表示，适合人阅读。本讲不用你会写完整 C 程序，只需要逐渐认出变量、运算和指针。

CPU 是执行指令的部件。寄存器在 CPU 内部，数量少，每个能保存固定宽度的位；内存可以先看成一排有编号的小格子，每格保存 **1 字节 = 8 位**。地址是格子的编号，内容是格子里存的位。编号和内容可能长得一样，作用却不同。

图 D1–F1：读一个内存中的整数。



这张图的意思是：**用寄存器rdi中保存的地址找到内存，把从那里读到的4字节数据复制到寄存器eax中。** `%rdi`、`%eax` 是CPU内部存储位置的名字，前面的 `%` 是本套汇编语法的寄存器标记，不是求余。`movl` 中的 `mov` 表示复制，最后的小写字母L表示处理4字节（32位）；AT&T写法中左边是来源，右边是目的。括号 `(%rdi)` 表示“把rdi中的数当地址，到内存读取”，没有括号的 `%rdi` 才表示直接使用寄存器里的数。

本例rdi保存 `0x1000`，所以连续读 `0x1000`～`0x1003` 的四个格子。`0x` 表示十六进制；内容栏的 `05`、`00` 也按十六进制读。四个字节共同表示一个32位整数。**小端**指权重最低的字节放在最低地址：本例从低地址到高地址的字节为 `05 00 00 00`，其权重依次是1、256、256^2、256^3，所以数值为 $5\times1 + 0\times256 + 0\times256^2 + 0\times256^3 = 5$。按通常高位在左的写数方式，就是 `0x00000005`。这里的“还原”是按规则解释数据，不会重新排列内存。

执行后eax得到5，rdi仍是地址 `0x1000`，内存中的四个字节也不变，因为读取复制不会清空来源。字节序与寄存器的更多规则分别在第7、8节展开。

初学补充：Q004：图中的movl、寄存器与“小端还原”是什么意思？ 从“格子的编号”和“格子里的内容”开始，逐项拆解这张图。

解释补全（2026-10-07）：补充第一张图所需的汇编符号、读取宽度、小端读法与执行前后状态，原图数据和结论不变。

对应 C 可以写成 `int *p = ...; int x = *p;`。`int *p` 表示 p 是指向 int 的指针；`*p` 表示读取它指向的 int；`&x` 表示 x 的地址。声明中的 `*` 和表达式中的乘号不能混看。指针保存地址，但指针的类型还告诉编译器：一次读多少字节、指针加 1 跨过多少字节。

本套讲义统一采用课程的 **x86-64、Linux、AT&T 汇编语法**。常见基本类型宽度如下；这是本课程环境，不是所有 C 实现都必须如此。

类型	字节数	位数	今天先记住什么
`char`	1	8	普通 char 是否有符号依实现；需要明确时写 signed/unsigned char
`short`	2	16	整数运算前常先提升为 int
`int`、`unsigned int`	4	32	有无符号改变解释及部分运算规则
`long`、`unsigned long`	8	64	本课程的 Linux x86-64 采用此宽度
指针，例如 `int *`	8	64	指针大小与它指向的 int 大小不同
`float` / `double`	4 / 8	32 / 64	下一讲学习；不能按补码整数解释其数值

1.2 必须分开的三层问题

一道题有时问机器寄存器结果，有时问 C 表达式。这不是同一层。

1. **数学整数**没有本题规定的位数限制，例如 127 + 1 = 128。
2. **定宽机器位运算**只保留指定宽度。8 位加法保存低 8 位，结果的位串再按题目要求解释。
3. **C 语言表达式**先按类型、整数提升和语言规则求值。无符号运算按模回绕；有符号溢出不能普遍当作“自动回绕”，它是未定义行为。

“未定义行为”不是某个特殊数值，而是语言不再保证程序有什么结果。考试若明确要求“按 w 位补码机器运算”，就计算该机器位模式；若问标准 C 程序是否总成立，就不能拿机器回绕替 C 作保证。本讲凡采用补码位模型或负数算术右移，都明确说明。

2. 二进制和十六进制：先会读位串

2.1 每一位都有自己的权重

十进制的 305 是 `3×100 + 0×10 + 5`。二进制的 `101101` 是 `1×32 + 0×16 + 1×8 + 1×4 + 0×2 + 1 = 45`。最右边是第 0 位，权重为 1；往左依次是 2、4、8、16。

十六进制使用 `0–9`、`A–F`，其中 A=10，…，F=15。因为 $16=2^4$，**一位十六进制恰好对应四位二进制**。写 `0x2D` 时，`0x` 只是表示后面采用十六进制，不是数值的一部分。

十六进制	0	1	2	3	4	5	6	7
二进制	0000	0001	0010	0011	0100	0101	0110	0111
十六进制	8	9	A	B	C	D	E	F
二进制	1000	1001	1010	1011	1100	1101	1110	1111

`1011 0110` 按每四位一组就得到 `0xB6`。反过来，`0x03A7` 是 `0000 0011 1010 0111`。不要先转十进制再绕回去。补前导零不改变无符号数值，但在定宽题里能帮助你看清最高位。

常用数量：2^8=256，2^{10}=1024，2^{16}=65536，2^{32}=4294967296。遇到很大的数，不必硬写十进制，`2^32 − 24` 往往更清楚。

2.2 位串本身没有“天生的正负”

同一位串 `1110 1011`，按 8 位无符号整数解释是 235，按 8 位补码解释是 −21。改变的是解释方式，不一定是存储的位。后面遇到强制转换、联合、浮点位模式时，都要先问：“改的是数值类型，还是只换了读法？”

3. 位运算和逻辑运算：长得像，处理对象不同

`& | ^ ~` 分别表示逐位与、或、异或、取反。它们对每一位分别操作。与要求两位都是 1；或要求至少一位是 1；异或要求两位不同。逻辑运算 `&& || !` 先把整个操作数判断为“零或非零”，结果是 int 类型的 0 或 1。

用 8 位模式演示：

按列看：每次只处理对应的两个位

a	0	0	1	1	1	1	0	0	输入
b	0	0	0	0	1	1	1	1	输入
a & b	0	0	0	0	1	1	0	0	两边都是 1
a | b	0	0	1	1	1	1	1	1	至少一边是 1
a ^ b	0	0	1	1	0	0	1	1	两边不同
~a	1	1	0	0	0	0	1	1	只展示 8 位取反

位串从左到右是第 7 位到第 0 位。这里是 8 位模式示意；真实 C 的 ~a 还要先检查整数提升。

但 `a && b` 是 1，因为两者都非零；`!a` 是 0；`!!a` 是 1。`~a` 不等于 `!a`。真实 C 中若 a 是 `unsigned char`，`~a` 之前通常先把 a 提升为 32 位 int，因此结果并不只是 8 位；想只保留低 8 位，需要明确截断，如 `(unsigned char)~a`。

逻辑运算还有**短路**：`A && B` 若 A 为假，不计算 B；`A || B` 若 A 为真，不计算 B。因此：

```
int d = 0;
int ok = (d != 0) && (12 / d > 2);
```

左边为假，右边除法不执行，ok 为 0。改成按位 `&` 就没有这种保护，两边都会求值，整数除零会造成未定义行为。后面解释条件传送是否适用时，还会用到这个思想。

掩码（mask）是一串配合位运算、控制哪些位保留或改变的0和1；它本身也是普通整数，不是特殊类型。 例如，只看8位模式，`1010 1101 & 0000 1111 = 0000 1101`：与1得到原位，与0得到0，所以掩码 `0000 1111` 让低4位留下、高4位清零。

下面按本课程32位 `unsigned int` 的x解释；常量后缀 `u` 表示无符号，`~` 是逐位取反，`<< k` 是左移k位。位编号从最右边的第0位开始，`0 ≤ k < 32`。

表达式	掩码怎样构造	结果
`x & 0xFFu`	`0x000000FF`，低8位是1，其余是0	保留低8位，高24位清零
`x & ~0xFu`	对 `0x0000000F` 逐位取反，得到 `0xFFFFFFF0`	低4位清零，其余保留
`x | (1u << k)`	从1左移k位，只有第k位是1	第k位设为1，其余不变
`x ^ (1u << k)`	同上，只有第k位是1	第k位翻转（0变1、1变0），其余不变

掩码中1的含义取决于操作：与 `&` 表示保留，或 `|` 表示设为1，异或 `^` 表示翻转。这里 `^` 不是乘方。这些表达式只计算结果，要改变x本身还需赋值，例如 `x = x & 0xFFu;`；“保留低8位”也不意味着把结果类型变成8位，结果仍为32位无符号整数。

初学补充：Q005：掩码是什么，四种掩码表达式分别怎样工作？ 用逐位对齐的例子拆解保留、清零、置1与翻转。

解释补全（2026-10-07）：补充掩码定义、常量后缀与位编号，并明确“设为1”和“翻转”的区别；原四种表达式的结论不变。

4. 无符号与补码：理解负数为什么这样存

4.1 用最高位的负权重理解补码

图 D1–F2：8 位模式 `1101 1011` 的两种解释。

同一位模式的两种数值解释

8 位模式保持一致；只有最高位的权重由 +128 变成 −128。

位位置	7	6	5	4	3	2	1	0
位值	**1**	**1**	**0**	**1**	**1**	**0**	**1**	**1**
无符号权重	128	64	32	16	8	4	2	1
补码权重	**−128**	64	32	16	8	4	2	1

无符号解释 **219**　　8 位补码解释 **−37**

补码加权　`−128 + 64 + 16 + 8 + 2 + 1 = −37`

原始模式为 1101 1011：无符号为 219，补码为 −37。最高位为 1 时，补码值等于无符号值减 256。

> 对照：只把最高位由 1 改成 0，位串成为 01011011，两种解释都等于 91。

读图时只取位值为 1 的列并累加权重。无符号结果是 219；补码结果是 −128+64+16+8+2+1=−37。只有最高位权重不同，因此最高位为 1 时，**补码值 = 无符号值 − 2^w**；最高位为 0 时两种解释相同。

w 位表示	最小值	最大值
无符号	0	2^w−1
补码	−2^(w−1)	2^(w−1)−1

8 位补码范围为 −128 到 127，比正数多一个负数。因此“最小负数取相反数”无法用同宽补码表示。例如 32 位 int 的最小值取负在 C 中会溢出；不要背成“所有负数都能安全取反变正”。

D1–E1：把 −37 写成 8 位补码，再读回来

助手补充，完整例题。

1. 正数 37 的 8 位表示是 `0010 0101`，即 `0x25`。
2. 对 8 位取反，得到 `1101 1010`；再加 1，得到 `1101 1011`，即 `0xDB`。
3. 反查：0xDB=219，最高位为 1，所以补码值是 219−256=−37。

“取反加一”是在固定位宽里构造相反数的便捷方法，不是负号占一位、剩下七位写绝对值。`1000 0001` 在 8 位补码中不是 −1。解题时写清位宽，再使用这条方法。

初学补充：Q001：为什么补码不是“负号加绝对值”？ 用位权表解释 `1000 0001` 为什么是 −127、−1 为什么写作 `1111 1111`，以及“取反加一”和位宽的含义。

4.2 扩展与截断：先决定保留什么

把数放进更宽的容器时，通常希望保留原数值。无符号数高位补 0；补码数复制原最高位，这叫符号扩展。例如 8 位 −37 扩展到 16 位是 `1111 1111 1101 1011`，即 `0xFFDB`。若补 0 得到 `0x00DB`，数值就变成了正数 219。

截断到更窄的 w 位则是保留低 w 位，之后再按目标类型解释。**截断不保证原数值不变。** 例如 `0x12DB`（4827）截到 8 位，位模式是 `0xDB`：无符号解释为 219，补码解释为 −37。这个**位模型**很好计算；但 C 语言把超范围整数转成有符号窄类型时，在教材所依据的传统 C 规则下，结果由实现定义，或引发实现定义的信号，不能对所有实现一概保证低位补码解释。本课程机器题采用通常的 x86 补码截断模型，题面若指定其他规则则按题面。

初学补充：Q002：截断为什么会改变数值？C、实现定义和 x86 截断模型是什么意思？ 从丢掉高位的示意图开始，分别解释机器位操作与C语言转换规则。

更正与澄清（2026-10-07）：明确截断不保证数值不变；补全传统C规则允许引发实现定义信号的情况。原例数值不变，核对依据见Q002。

扩大位宽和改变有无符号若同时出现，应按实际转换顺序处理。例如一个值为 −37 的 signed char 转成 unsigned int，是把**数值 −37 转成 32 位 unsigned**，得到模 2^{32} 的结果；不是先偷偷转成 unsigned char 的 219。若代码真的写了 `(unsigned char)c`，那才先变成 219。

5. C 类型转换：先定类型，再算数

5.1 整数提升不是可忽略的小细节

比 int 小的整数类型参与多数算术、位运算、比较前，先做整数提升。本环境里 int 能容纳所有 unsigned char 和 unsigned short 的值，所以它们也提升为 int。`unsigned char a=250, b=10;` 中，`a+b` 先得到 int 类型的 260。

如果再把260赋给 `unsigned char`，就要转换成只能表示0～255的8位无符号整数。C对这种无符号整数转换规定按模256处理。**本例中，“模256”就是除以256取余数：260 = 1×256 + 4，所以转换后得到4。** 这里用256，是因为8位共有2^8=256种位模式。这与只保留低8位的结果相同：260写成 `1 | 0000 0100`，竖线左边的1代表256，丢掉后留下的 `0000 0100` 就是4。加法的结果仍是260，转换到窄类型后才得到4。

初学补充：Q003：“模”是什么意思，为什么这里要模256？ 从除法的余数开始，把取模、低8位和整数提升的先后顺序接起来。

解释补全（2026-10-07）：原文直接使用“模256”，遗漏了术语含义及与8位范围的联系；现已补充，原计算结果不变。

这解释了为什么不能看见“两个 8 位变量相加”就断言 C 在 8 位中做加法。汇编中的 **`addb` 是8位加法指令：`add` 表示加，`b` 表示byte，即1字节（8位）**。例如AT&T写法 `addb $10, %al` 表示把常数10加到8位寄存器al中，结果写回al；若al原位模式按无符号解释为250，则只保存和的低8位，得到4。它直接指定8位机器加法，与C中先提升成int计算是不同层次。

初学补充：Q006：addb是什么，和C里的加法有什么区别？ 拆解指令后缀、操作数和250＋10的位级结果。

5.2 混合比较先做公共类型转换

在 32 位 int 与 unsigned int 的常见组合中，int 会转为 unsigned int。负数因此变成一个大的非负数。例如 `-2 < 1U` 比较时，左边按 32 位 unsigned 解释，结果为假。

不要扩展成“只要有 unsigned，整个表达式一定无符号”。本环境的 64 位 long 能容纳全部 32 位 unsigned int，所以 `-2L < 1U` 会先把右边转换为 long，结果为真。够用的判断顺序是：

1. 先提升 char、short 等窄类型。
2. 类型一样，直接按该类型计算。
3. 都有符号或都无符号时，转到等级更高的类型。
4. 一有符号一无符号：若无符号类型等级不低，转到无符号类型；否则检查**等级更高的有符号类型**能否表示无符号类型的全部值。能，则统一到这个有符号类型；不能，则双方统一到**这个有符号类型对应的无符号类型**。

等级与位宽不是同一概念：常见等级为 `int < long < long long`，每种有符号类型与其对应无符号类型等级相同。本环境中long与long long都是64位，long long的等级仍更高。判断能否容纳时看**整个类型范围**，不是当前变量的值。例如long能容纳32位unsigned int，二者统一为long；long long不能容纳64位unsigned long的全部值，二者统一为unsigned long long。

初学补充：Q007：无符号类型等级较低时，究竟转成什么？ 展开“能装下”和“装不下”两条分支，区分等级、位宽与取值范围。

今天优先练熟 `int/unsigned int` 和 `long/unsigned int`。**整数常量只有U或u后缀时，依次从unsigned int、unsigned long、unsigned long long中选第一个能表示其值的类型**，候选中没有char或short。本环境中 `40U` 是unsigned int，`4294967296U` 则是unsigned long。没有后缀的小十进制常量通常是int；其他后缀或进制要按各自候选表判断。声明中的 `unsigned` 单独写是 `unsigned int` 的简写，与“U后缀常量可能选择不同类型”是两件事。

初学补充：Q008：整数常量只有u后缀，怎样判断具体类型？ 对照边界例子，以及常量类型、变量类型和unsigned简写。

更正与澄清（2026-10-07）：将“较宽的有符号类型”改为“等级更高的有符号类型”，避免混淆等级与位宽；补全仅U后缀的候选顺序。规则依据见Q007、Q008。

D1–E2：负数变 unsigned 后，为什么加法和比较看起来反常?

改编自 2025 期末无答案版（来源资料未随仓库分发），第一题第 1 小题，PDF 第 2 页。将原数值改为 −24 和 40，并把选择题改成逐步推导；不沿用原题评分。

```
int x = -24;
unsigned u = (unsigned)x;
unsigned v = u + 40U;
```

读代码提示：这里 `unsigned` 就是 `unsigned int`，变量名u只是名字；`40U` 因为40能被unsigned int表示，也是unsigned int。Q007：混合类型转换分支 · Q008：U后缀常量的具体类型。

求 u 的位模式、v，以及 `x < u`、`x == u` 的结果。

第一步：明确宽度。 int 和 unsigned 均为 32 位。负数转 unsigned 的值是对 2^{32} 取模，所以 $u=2^{32}-24$，十六进制为 `0xFFFFFFE8`。

第二步：在 unsigned 中加。 $u+40=(2^{32}-24)+40=2^{32}+16$。无符号结果保留模 2^{32} 的值，所以 v=16。不是算术坏了，而是有限范围的无符号加法就这样定义。

这里$2^{32}+16$是推导中的**数学和**，不是一个32位int已经保存的值；操作数u与40U都是unsigned int，因此本步是无符号运算。若换成C有符号int运算，必须检查实际运算类型的溢出，不能沿用本步保证。参见 Q009：数学结果达到$2^{32}+16$时如何处理 中的追问2。

第三步：比较前统一类型。 `x < u` 中 x 被转成 unsigned，转换后的 x 恰好等于 u，因此小于为假，等于为真。不能一边坚持 x 是负数，一边拿 u 的无符号值直接比较。

一个稳定的作答格式是：**原类型 → 转换后的类型与数值 → 运算 → 结果类型**。只写最终“真/假”，即使猜对也不容易发现思路哪里不稳。

初学补充：Q009：有符号结果超出范围时，也能直接取模吗？ 先核对无符号转换的理解，再区分补码机器运算、C有符号算术与超范围类型转换。w位补码的范围是−2^(w−1)～2^(w−1)−1，不是以2^w作为正负边界。

6. 运算、溢出与移位

6.1 位宽限制如何改变结果

无符号 w 位加法、乘法都可用“数学结果对 2^w 取模”计算。以 8 位位模型为例，240+32=272，保留低 8 位得到 16；20×20=400，保留低 8 位得到 144。

相同位加法结果换成补码解释，可能发生有符号溢出：120+20 的数学结果为 140，8 位结果为 `1000 1100`，补码解释是 −116。这里说的是 **8 位机器加法**。若写成 C 的 `int a=120; a+20`，32 位 int 完全装得下，结果就是 140；即使 a 是 char，整数提升也常使加法在 int 中进行。

判断补码加法溢出：两个正数相加得到负数，或两个负数相加得到非负数。异号相加不会产生有符号加法溢出。无符号进位和有符号溢出是两个问题：同一个位加法可以只有其中一种，也可以两种都有。下一讲用 CF、OF 给它们命名。

若题目明确要求**两个w位补码数的机器加法**，数学和超过最大值2^(w−1)−1时，正溢出的机器结果按补码读为“数学和−2^w”；例如8位的140变为140−256=−116。若问C中实际w位有符号加法，这种越界是未定义行为，不能保证回绕结果；先检查整数提升，别把变量原位宽误当成运算位宽。

针对“负方向超范围”和“C能否直接按机器回绕”的解释，见 Q009：有符号结果超出范围时，也能直接取模吗？ 。其中追问2、3分别计算数学结果$2^{32}+16$与$-2^{32}-6$只保留低32位后如何按补码读取。

6.2 左移、逻辑右移、算术右移

先用8位机器模式 `1101 1011` 观察移两位。**下面三行分别从同一个初态出发，不是连续执行，也没有给这串位统一指定C类型。** 原模式按8位无符号数读是219，按8位补码读是−37。

操作	得到的8位模式	空出来的位怎么填	按无符号读	按补码读
左移2位	`0110 1100`	右侧补0，超出左边的位丢弃	108	108
逻辑右移2位	`0011 0110`	左侧补0	54	54
算术右移2位	`1111 0110`	左侧复制原最高位；本例为1	246	−10

有符号整数类型允许表示负数、零和正数；**补码**是这些整数的一种二进制表示方法。本课程x86-64的有符号整数采用补码，因此“按补码读”就是按相应位宽的有符号整数读。“有符号”不等于当前值为负，“补码”也能表示正数和零。逻辑右移通常配合无符号解释，算术右移通常配合补码有符号解释；具体机器指令按指定规则移动位，不会自动知道C类型。

初学补充：Q010：移位表中是有符号还是无符号？补码与有符号有什么区别？ 对照原位模式的两种读法及三种移位结果。

解释补全（2026-10-07）：明确三行独立、区分位操作与数值解释，并补上两种读法的十进制值；原移位结果不变。

机器的左移可以用乘 2^k、保留低位理解。无符号逻辑右移相当于除以 2^k 再向下取整。补码算术右移也相当于向下取整，而 C 有符号整数除法从 C99 起采用**向零截断**，负数时二者可能不同。

D1–E3：为什么 −19 除以 8，不能直接照搬右移?

助手补充，按 32 位补码且负数算术右移的课程机器假设。

1. 精确商为 −2.375。向零截断是 −2，所以 C 的 `-19 / 8` 为 −2。
2. 算术右移 3 位相当于向下取整，得到 −3。它向更负的方向走了一步。
3. 对负数先加 2^3−1=7：−19+7=−12；再算术右移 3 位，得到 −2。

这多加的 7 称为偏置 bias：把负数所在的区间挪到能够向零截断的位置。一般除以 2^k 时，非负数直接右移；负数先加 2^k−1，再算术右移。

```
/* 仅采用本课程负数算术右移假设；这里限制 0 <= k <= 30。 */
int div_power2(int x, int k) {
    int bias = (1 << k) - 1;
    return (x + (x < 0 ? bias : 0)) >> k;
}
```

`条件 ? A : B` 表示条件为真取 A，否则取 B，只求值被选的一支。限制 k 是为了避免构造 bias 时让 `1 << k` 本身越过有符号可表示范围；负 x 加上这范围内的非负 bias 不会发生有符号加法溢出。

这里右移的左操作数 `x + (x < 0 ? bias : 0)` 是有符号int。按代码注释采用的课程实现假设，**若这个和为负，右移自动在左侧补1；非负则补0，不需要手动补符号位**。复制的是相加后实际被右移值的最高位。bias的作用是修正“向下取整”与“向零截断”的差别，不是实现补位；例如−19先加7得到−12，再算术右移3位得到−2。

初学补充：Q011：C代码中的右移会自动补符号位吗？bias是用来补位的吗？ 区分自动补位、左操作数类型与除法取整修正；传统C对负数右移的实现前提见6.3。

6.3 哪些移位规则不能混用

情境	应怎样作答
C 无符号左移	合法移位量内，按乘 2^k 后模 2^w；w 取提升后左操作数宽度
C 无符号右移	左补 0，相当于向下取整除以 2^k
C 有符号负数右移	教材所用传统 C 规则中是实现定义；本课程 x86 通常采用算术右移
C 对负有符号数左移，或非负有符号左移结果不可表示	按教材所用传统 C 规则，不作可靠回绕结果，属于未定义行为
C 移位量为负，或不小于提升后左操作数宽度	未定义行为；`1U << 32` 不能说等于 1 或 0
x86 指令 `shl/sal`、`shr`、`sar`	直接按机器指令追踪；sal/shl 左移，shr 逻辑右移，sar 算术右移

x86 的移位指令对移位计数有硬件处理规则，例如 32 位操作取计数低 5 位，64 位操作取低 6 位。**这不使越界移位的 C 表达式变得有定义。** 题目是 C 还是已经给定汇编，要先看清。

7. 字节序：变的是存放顺序，不是数值

图 D1–F3：把 32 位数 `0x1234ABCD` 放在地址 0x2000。



读图从地址最小的一格开始。小端把**最低有效字节**放在**最低地址**，x86–64 使用小端。大端把最高有效字节放在最低地址。两种表示存的整数都还是 `0x1234ABCD`，不会因为小端就变成 `0xCDAB3412`；后者只是把按地址列出的字节串误当成了通常书写的数。

拆字节时先从右往左每两位十六进制一组，再按地址放好。不要把每个字节内部的位再反转，也不要把每个十六进制字符独立反转。

考题写“第一个字节”时最好明确写“最低地址处字节”；写“最高字节”则明确是最高有效字节。符号扩展和字节序也要分步：先计算完整新位模式，再把它分成字节按小端排列。

8. 汇编入门：先读宽度，再读源与目的

8.1 寄存器名称中的宽度

寄存器不自带 C 类型。`%rax` 是 64 位寄存器；`%eax`、`%ax`、`%al` 是它的一部分，不是四个互不相干的盒子。

图 D1–F4：同一个 rax 的不同访问范围。



读图时以右端第 0 位对齐：eax 是低 32 位；ax 是低 16 位；al 是低 8 位；ah 是第 8—15 位。ah 只需认得，现代 64 位代码常用低字节寄存器，不必把它当主要工作对象。

64 位	低 32 位	低 16 位	低 8 位
rax	eax	ax	al
rbx	ebx	bx	bl
rcx	ecx	cx	cl
rdx	edx	dx	dl
rsi / rdi	esi / edi	si / di	sil / dil
rsp / rbp	esp / ebp	sp / bp	spl / bpl
r8—r15	r8d—r15d	r8w—r15w	r8b—r15b

一个高频规则：**写 32 位通用寄存器会把对应 64 位寄存器的高 32 位清零**。写 8 位或 16 位子寄存器通常保留其余位。例如原 rax=`0xAABBCCDD11223344`：

- `movb $0x77, %al` 后为 `0xAABBCCDD11223377`。
- 从原状态执行 `movw $0x7788, %ax` 后为 `0xAABBCCDD11227788`。
- 从原状态执行 `movl $0x778899AA, %eax` 后为 `0x00000000778899AA`。

以上三行彼此独立，不是连续执行。读 eax 不会清高位，**写 eax 才会**。

“读eax”是取出它的现有内容来用，“写eax”是把结果存进它。例如 `movl %eax, %ecx` 读eax、写ecx，不改变rax，但会清零rcx的高32位；`movl $5, %eax` 写eax，使完整rax变成5。即使 `movl %eax, %eax` 把原值写回，仍然发生了写eax，所以也会清零rax高32位。判断依据是有没有写入，不是低32位的数值是否改变。

初学补充：Q012：读eax与写eax有什么区别，为什么只有写才清高位？ 用同一初态分别追踪读取、写入和原值写回。

8.2 读 AT&T 的一条指令

`movq %rdi, %rax` 的方向是 **源在左，目的在右**，把 rdi 的 64 位内容复制到 rax。mov 不会把源清空，也不是交换。后缀 b/w/l/q 对应 1/2/4/8 字节；这里 l 表示 32 位，与 C 的 long 在本环境为 64 位并不矛盾。

操作数	示例	含义
立即数	`$8`、`$0x2000`	直接使用这个常数
寄存器	`%rax`	读取或修改寄存器的位
内存	`(%rdi)`、`8(%rdi)`	先算地址，再按指令宽度访问内存

`movq $0x2000, %rax` 把数 0x2000 放进 rax；`movq (%rdi), %rax` 才去内存取数。普通 mov 的源和目的不能同时是内存，要借一个寄存器中转。立即数也不能成为目的。

注意：`%rdi` 表示寄存器操作数，里面的值可以是地址，也可以是普通整数；不能直接把它定义成“地址”。在这里的mov指令中，`(%rdi)` 表示以rdi的值为地址的内存操作数，在源位置读内存，在目的位置写内存。初学补充：Q013：(%rdi)和%rdi的区别，后者一定是地址吗？ 。

两操作数的算术指令同样读成 `目的 = 目的 运算 源`：`subq %rcx,%rax` 是 rax−rcx；`addq` 相加，`imulq` 的双操作数形式留下乘积的低位，`andq/orq/xorq` 做位运算。这些指令还会影响条件码，今天先追踪数据，下一讲学习条件码。

8.3 复杂寻址只是一个加法公式

`D(Rb,Ri,S)` 表示地址 `D + Rb + Ri×S`。D 是字节位移；Rb 是基址寄存器内容；Ri 是索引寄存器内容；S 是比例因子，允许 1、2、4、8。省略基址或索引时对应项为零，省略比例通常按 1。

比例因子不是由 mov 的后缀自动决定的。 `movq 8(%rdi,%rcx,4),%rax` 仍然先算 `rdi+4×rcx+8`，然后从那里读 8 字节。

D1–E4：相同的括号，mov 和 lea 做的事为什么不同?

助手补充。 已知 rbx=0x3000、rcx=3，地址 0x3020 开始的 8 字节按小端表示整数 9。分别从这个初始状态执行：

```
movq 8(%rbx,%rcx,8), %rax
leaq 8(%rbx,%rcx,8), %rax
```

第一步都计算 `0x3000 + 3×8 + 8 = 0x3020`。随后 mov 去地址 0x3020 读取 8 字节，rax 得到 **9**；lea 直接把计算得到的地址值 **0x3020** 写入 rax，不读这块内存。

lea 不要求算出的数一定是有效内存地址，所以还能用于普通算术：`leaq (%rdi,%rdi,4),%rax` 得到 5×rdi 的低 64 位。括号在 lea 中给出的是计算公式，不是访存命令。lea 也不修改条件码，后面追踪比较时尤其要记住。

8.4 扩展指令：是补零还是补符号位?

movz 系列零扩展，movs 系列符号扩展。名字中最后两个字母通常给源宽度和目标宽度：`movzbl` 为 byte→long，即 8→32 位；`movsbq` 为 8→64 位；`movslq` 为 32→64 位。

假设 al 的位模式是 `0x90`：

指令，分别执行	执行后 rax	为什么
`movzbl %al,%eax`	`0x0000000000000090`	先 8→32 零扩展，再由写 eax 清高 32 位

指令，分别执行	执行后 rax	为什么
`movsbl %al,%eax`	`0x00000000FFFFFF90`	先 8→32 符号扩展，再清高 32 位
`movsbq %al,%rax`	`0xFFFFFFFFFFFFFF90`	直接 8→64 符号扩展

第二行最容易错：它确实做了符号扩展，但**目标只有 32 位**；随后写 eax 的规则仍然有效。32→64 零扩展常直接用 `movl` 写 32 位目标实现，不需要寻找不存在的普通 `movzlq` 指令。

D1–E5：逐行追踪必须保留中间状态

助手补充。 rdi=0x4000，地址 0x4000 的 4 字节表示 `0xFFFFFFF9`，初始 rax 的值不重要。

```
movl   (%rdi), %eax
movslq %eax, %rax
addq   $4, %rax
```

行	源是什么	执行后 rax	改了哪里
1	地址 0x4000 的 4 字节	`0x00000000FFFFFFF9`	写 eax，连带清 rax 高位；内存不变
2	eax 的 32 位补码值 −7	`0xFFFFFFFFFFFFFFF9`	符号扩展到整个 rax
3	立即数 4	`0xFFFFFFFFFFFFFFFD`，补码值 −3	修改 rax；内存不变

第一行之后，完整 rax 按 64 位补码解释仍是正数；第二行之后才是 64 位 −7。不要把“低 32 位可解释为 −7”与“整个寄存器就是 −7”混成一句话。

9. 独立练习：6 道短题 + 1 段追踪

以下 D1–P1—P7 为必做，共 **25 分，20 分为 80% 过关线**。P1—P4 是整数短题，P5—P6 是汇编短题，P7 是短追踪。每题写出关键过程；得到提示后完成的部分记为练习，不计入首次独立成绩。题目默认沿用本讲环境，所有内存地址均假定合法。

D1–P1：同一模式、两种解释与字节序（4 分）

助手补充。

1. 把 8 位 `0xB6` 写成二进制。（1 分）
2. 分别按 8 位无符号数、8 位补码解释，给出十进制值。（各 1 分）
3. 32 位数 `0x81A2C3D4` 存在地址 0x5000，请画出小端环境中 0x5000—0x5003 的四个字节。（1 分）

D1–P2：位运算、逻辑运算与短路（3 分）

助手补充，a、b 均为 unsigned int，a=0x28，b=0x14。

1. 写出 `a & b` 与 `a && b` 的值。（合计 1 分）
2. `int z=0; int r=(z!=0)&&(18/z>1);` 是否执行除法？r 为多少？（1 分）
3. 用一个按位与表达式将 unsigned int x 的低 4 位清零，其余位不变。（1 分）

本题追问：Q005：0x0u的写法合法吗，能用于这道题吗？（含第3小题解答，按需打开）。

D1–P3：提升、扩展和比较（3 分）

助手补充。

1. `signed char c=-18; int a=c;` 写出 a 的 32 位十六进制表示。(1 分)
2. `unsigned char p=240, q=30; int t=p+q;` 写出 t 的值，并说明在哪种类型中相加。(1 分)
3. 判断 `-3 < 2U` 和 `-3L < 2U`，各说明比较时采用的公共类型。(合计 1 分)

D1–P4：移位与运算层次（4 分）

助手补充。

1. 对 8 位机器模式 `1010 1100` 分别逻辑右移 2 位、算术右移 2 位，写出结果位模式。(合计 1 分)
2. 计算 C 的 `-25 / 8` 和本课程机器假设下的算术右移 `-25 >> 3`。若用 bias 修正后者，应先加多少，修正结果是多少？ (2 分)
3. 32 位环境下，C 的 `1U << 32` 能否保证结果为 1？写出依据。(1 分)

D1–P5：子寄存器写入（3 分）

助手补充。以下三问**分别**从 rax=`0x89ABCDEFFEDCBA98` 开始，写出执行后的完整 rax。

1. `movb $0x11, %al`。(1 分)
2. `movl $0x76543210, %eax`。(1 分)
3. `movsbl %al, %eax`。(1 分)

D1–P6：地址还是内容（3 分）

助手补充。rbx=0x200，rcx=3；地址 0x228 开始的 8 字节表示整数 0x8899。以下分别执行，写出 rax 的值，并注明是否读取数据内存。

1. `leaq 0x10(%rbx,%rcx,8), %rax`。(1 分)
2. `movq 0x10(%rbx,%rcx,8), %rax`。(1 分)
3. `movq $0x228, %rax`。(1 分)

D1–P7：5 行追踪（5 分）

助手补充。初始 rdi=0x1000，rsi=2；地址 0x1008 的 4 字节表示 `0x00000007`，地址 0x100C 的 4 字节表示 `0xFFFFFFFE`。按顺序执行：

```
leaq   (%rdi,%rsi,4), %rdx
movl   (%rdx), %eax
addl   4(%rdx), %eax
movl   %eax, (%rdx)
movslq 4(%rdx), %rcx
```

每行 1 分：列出源值/有效地址、执行后改变的寄存器或内存及其值。凡写 eax，请报告完整 rax；最后一行报告完整 rcx。必须把第 4 行的写内存与前几行的读内存区分开。

10. 达标后的 2 道可选加练

以下不计入 25 分。如果未达标，先修复 P1—P7 的错因，再用它们做针对性练习。

D1–A1：边界与“看起来相同的代码”

助手补充。32 位 int 最小值记为 INT_MIN，最大值记为 INT_MAX。

1. C 表达式 `-INT_MIN` 是否能保证得到正数？说明原因。
2. `unsigned int a=0xFFFFFFFFU;` 时 `a+1U` 的值是多少？为什么与 `INT_MAX+1` 的语言保证不同？

3. `unsigned char a=255;` 时 `a+1` 的类型和值分别是什么？如果赋给 unsigned char b，b 又是多少？

D1–A2：清低位等价于向下对齐

改编自 2025 期末无答案版（来源资料未随仓库分发），第一题第 3 小题，PDF 第 2 页。把移位量 5 改为 3，并改为解释题。

对 32 位 unsigned int x，令 `y=(x>>3)<<3`。

1. 写出与 y 等价的位掩码表达式。
2. 说明为什么 `y<=x`，以及 `x-y` 的范围。
3. 用 x=77 验证上述结论；再判断 x 为 8 的倍数时是否必有 y=x。

11. 收束：做题时按这个顺序检查

整数题先标 **类型和宽度**，再写位模式或数值；转换题写清先提升、再统一类型、最后赋值；溢出题确认问的是机器位结果还是 C 保证。字节序题先求完整模式，再拆成字节按地址摆放。

汇编题每行问四件事：**操作宽度是多少？源在哪里？有效地址是多少？目的的哪些位或哪些字节改变？** lea 只计算，mov 才按操作数决定是否访问内存；写 eax 记得清高位。

本讲通过的实际标准是：P1—P7 达到 20/25，能用自己的话说明有符号/无符号混合比较，能画对一次小端字节图，P7 能区分读内存、写内存、只改寄存器。若分数达标但仍分不清地址与内容，先修这一点再进入条件与分支。只记录真实出现的错因，不预先填“我已掌握”。

12. 对照来源与回查位置

以下页码全部是 **PDF 文件页序**，不是教材印刷页码。阅读本讲不要求先通读原书；只有概念需要回查时才打开对应小节。

本讲内容	已核对的课件位置	教材对应小节
进制、数据宽度	第 2 讲 PDF 第 3—5 页	2.1.1—2.1.2
位运算、逻辑运算、移位	第 2 讲 PDF 第 7—12 页	2.1.6—2.1.9
补码、范围、转换、扩展截断	第 2 讲 PDF 第 14—34 页	2.2.1—2.2.7
加乘溢出、移位乘除	第 2 讲 PDF 第 36—48 页；bias 推导为教材补充	2.3.1—2.3.7，特别是 2.3.7
内存、地址、字节序	第 2 讲 PDF 第 55—64 页	2.1.3；硬件背景参见 1.4.1
C 到机器指令、数据宽度	第 4 讲 PDF 第 12—21 页	3.2—3.3
寄存器、mov、寻址	第 4 讲 PDF 第 26—40 页	3.4.1—3.4.3；扩展细节见 3.4.2
lea、算术、移位指令	第 4 讲 PDF 第 42—46 页	3.5.1—3.5.3

原始来源：第 2 讲课件（来源资料未随仓库分发）、第 4 讲课件（来源资料未随仓库分发）、教材原书第 3 版中文版（来源资料未随仓库分发）。教材引用到小节，不猜填印刷页码。

本讲未使用 2025 第一次小测的具体题面或答案。后续针对学习中真实问题的补讲另存文件，本讲保留为最初教学版本。